

Experimental Evaluation of Real-Time Multi-Block Gaussian Volume Rendering

1 Experimental Objectives

1.1 RQ1: Scalability of Active Gaussian Payload Across Multiple Blocks

1.1.1 Research Question

RQ1: How does active Gaussian payload affect steady-state rendering speed across multiple pretrained blocks on a fixed GPU?

1.1.2 Objective

This experiment evaluates how steady-state rendering performance changes as the active Gaussian payload increases. Unlike the preliminary single-block evaluation, the final experiment uses five independently pretrained blocks to measure whether the observed payload–performance relationship remains consistent across different Gaussian spatial distributions.

The experiment addresses payload scalability only. GPU-memory normalisation, cross-hardware comparison, and the derivation of a deployment threshold are reserved for subsequent research questions.

1.1.3 Experimental Setup

The evaluation was conducted using five pretrained Gaussian blocks:

- `gaussians_016`,
- `gaussians_017`,
- `gaussians_026`,
- `gaussians_027`, and
- `gaussians_028`.

Each block contained 50,000 Gaussians, and each Gaussian record occupied 44 bytes. The experiments were performed on an NVIDIA RTX 5000 Ada Generation GPU with approximately 32 GB of VRAM and CUDA compute capability 8.9.

All rendering parameters were held constant across blocks, payload levels, and repetitions. The fixed settings are summarised in Table 1.

Table 1: Fixed rendering settings for the multi-block RQ1 experiment.

Parameter	Value
GPU	NVIDIA RTX 5000 Ada Generation
Image resolution	128×128
Depth samples per ray	64
Vertical field of view	90°
Camera position	$(0, 0, 0)$
Camera rotation	$(0^\circ, 0^\circ, 0^\circ)$
Rendering bounds	$[-1, 1]^3$
Tile size	16×16
Mahalanobis cutoff	20
Warm-up frames	20
Measured frames	300
Timing repetitions	5
Gaussians per full block	50,000
Bytes per Gaussian	44

1.1.4 Payload Construction

For each block, one fixed shuffled Gaussian ordering was generated using a constant random seed. This prevents the experiment from depending on the original ordering of the Gaussian records.

The same ordering was reused for every payload level and every timing repetition. Consequently, the subsets for each block were nested:

$$\mathcal{G}_{10\%}^{(b)} \subset \mathcal{G}_{20\%}^{(b)} \subset \mathcal{G}_{30\%}^{(b)} \subset \dots \subset \mathcal{G}_{100\%}^{(b)}, \quad (1)$$

where b denotes a block.

The evaluated retention ratios were

$$\rho \in \{0.10, 0.20, 0.30, 0.40, 0.50, 0.60, 0.75, 1.00\}. \quad (2)$$

For each block and retention ratio, the active Gaussian population was

$$N_{\text{active}} = \rho N_{\text{full}}, \quad (3)$$

where

$$N_{\text{full}} = 50,000. \quad (4)$$

The raw active payload was calculated as

$$B_{\text{active}} = N_{\text{active}} \times 44 \text{ bytes}. \quad (5)$$

The resulting raw payload was approximately 0.210 MiB at 5,000 Gaussians and 2.098 MiB at 50,000 Gaussians.

1.1.5 Timing Protocol

Renderer construction and preprocessing were excluded from steady-state timing. The excluded stages were:

- Gaussian binary loading,

- subset generation,
- CUDA allocation,
- host-to-device transfer,
- Gaussian preprocessing,
- tile binning,
- prefix scan,
- Gaussian-tile duplication,
- radix sorting,
- tile-range construction,
- output download, and
- image and CSV writing.

For every block and payload level, 20 untimed warm-up frames were rendered. This was followed by 300 individually timed frames using CUDA events. Every configuration was repeated five times.

The total number of benchmark runs was therefore

$$5 \text{ blocks} \times 8 \text{ payload levels} \times 5 \text{ repetitions} = 200 \text{ runs.} \quad (6)$$

For each measured frame j , rendering speed was calculated as

$$\text{FPS}_j = \frac{1000}{T_j}, \quad (7)$$

where T_j is the frame time in milliseconds.

The primary outcome was median FPS. Median render time, fifth-percentile FPS, Gaussian-tile pair count, and variation across repeated measurements were also recorded.

1.1.6 Aggregate Results

Table 2 presents the mean performance across the five blocks at each payload level. Each reported value first aggregates the five timing repetitions for a block and then averages across the five blocks.

Table 2: Aggregate RQ1 results across five pretrained Gaussian blocks.

Retention	Active Gaussians	Payload (MiB)	Mean Tile Pairs	Mean Median Time (ms)	Mean Median FPS	Mean p5 FPS
10%	5,000	0.210	77,938	3.028	330.58	329.57
20%	10,000	0.420	156,358	6.047	165.58	165.46
30%	15,000	0.629	232,872	8.979	111.51	111.48
40%	20,000	0.839	309,243	11.903	84.11	84.08
50%	25,000	1.049	385,166	14.806	67.61	67.59
60%	30,000	1.259	462,053	17.758	56.36	56.36
75%	37,500	1.574	578,396	22.224	45.04	45.03
100%	50,000	2.098	768,733	29.554	33.87	33.86

The complete multi-block experiment contains 200 measurements covering all five blocks, eight payload ratios, and five repetitions.

1.1.7 Rendering Performance

Figure 1 shows the effect of active Gaussian count on rendering speed across all five pretrained blocks. FPS decreases monotonically as the active payload increases, with a similar trend observed for every block.

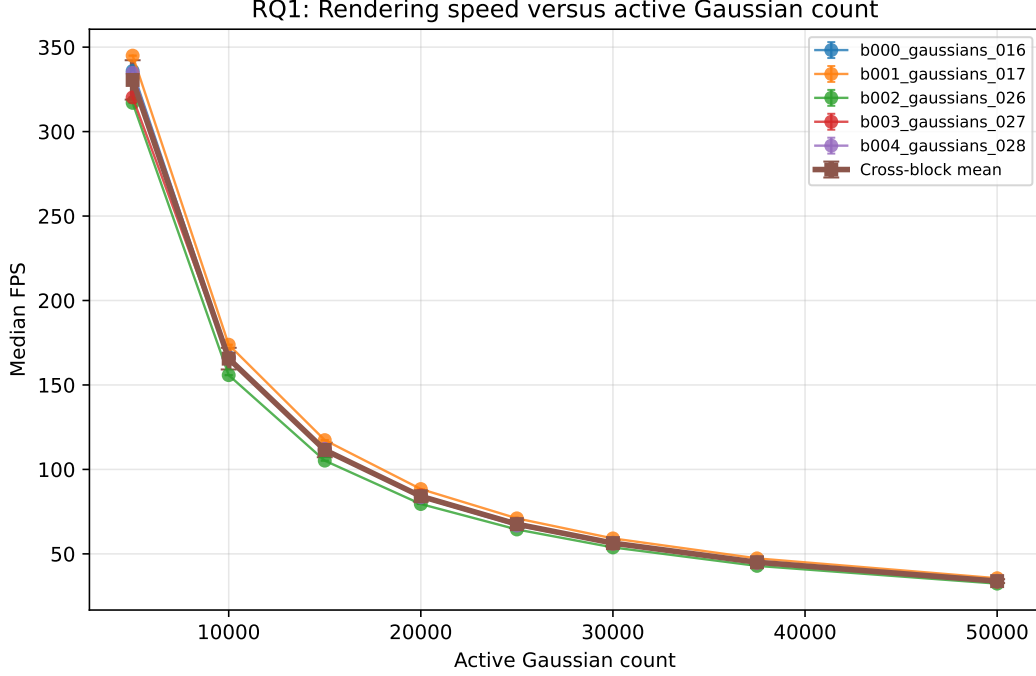


Figure 1: Median rendering speed as a function of active Gaussian count across five pretrained blocks. Each point represents the mean of the median FPS values obtained from five repeated runs, where each run contains 300 measured frames following 20 warm-up frames. Error bars on the individual block curves indicate variation across timing repetitions, while the aggregate curve shows variation across blocks.

1.1.8 Payload–Performance Relationship

Figure 2 presents the same experiment in terms of rendering time. Unlike FPS, frame time exhibits an approximately linear relationship with active Gaussian count, making this representation more appropriate for analysing computational scalability.

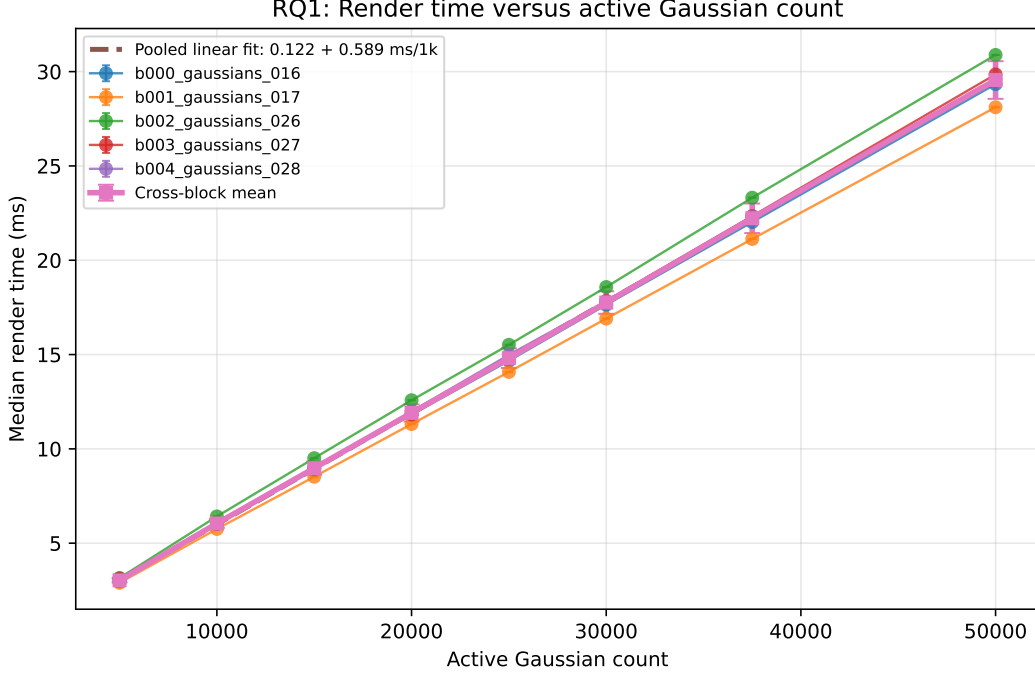


Figure 2: Median rendering time as a function of active Gaussian count for five pretrained blocks. Individual curves show block-specific results, and the thicker aggregate curve shows the cross-block mean. The dashed line represents the pooled linear fit. Rendering time increases approximately linearly with the number of active Gaussians.

Rendering time increased monotonically as the active Gaussian population increased. Across the five blocks, increasing the payload from 5,000 to 50,000 Gaussians increased the mean median frame time from approximately

$$3.03 \text{ ms} \tag{8}$$

to

$$29.55 \text{ ms.} \tag{9}$$

Over the same range, mean median rendering speed decreased from approximately

$$330.58 \text{ FPS} \tag{10}$$

to

$$33.87 \text{ FPS.} \tag{11}$$

Thus, increasing the active Gaussian population by a factor of ten produced an approximately tenfold increase in frame time.

The inverse relationship between FPS and rendering time explains the nonlinear FPS curve:

$$\text{FPS} = \frac{1000}{T_{\text{render}}}. \tag{12}$$

For example, increasing the Gaussian count from 5,000 to 10,000 approximately doubled the aggregate frame time from 3.03 ms to 6.05 ms and approximately halved FPS from 330.58 to 165.58.

1.1.9 Block-Level Results

Table 3 reports the performance of each block at the full 50,000-Gaussian payload.

Table 3: Block-level performance at the complete Gaussian payload.

Block	Gaussian–Tile Pairs	Median Time (ms)	Median FPS
gaussians_016	761,972	29.337	34.09
gaussians_017	736,767	28.111	35.57
gaussians_026	800,128	30.885	32.38
gaussians_027	773,308	29.854	33.50
gaussians_028	771,490	29.584	33.80

Although all blocks contained the same number of Gaussians and the same raw payload size, their rendering times differed. At the full payload, **gaussians_026** generated 800,128 Gaussian–tile pairs and rendered at approximately 32.38 FPS. In contrast, **gaussians_017** generated 736,767 pairs and rendered at approximately 35.57 FPS.

This result shows that Gaussian count alone does not completely determine rendering cost. The projected spatial coverage of the Gaussians, represented by the Gaussian–tile pair count, also affects the amount of work performed by the rendering kernel.

Although Gaussian count captures the dominant scalability trend, blocks with the same number of active Gaussians do not always require exactly the same rendering time. Figure 3 therefore relates frame time to the number of Gaussian–tile pairs generated during preprocessing.

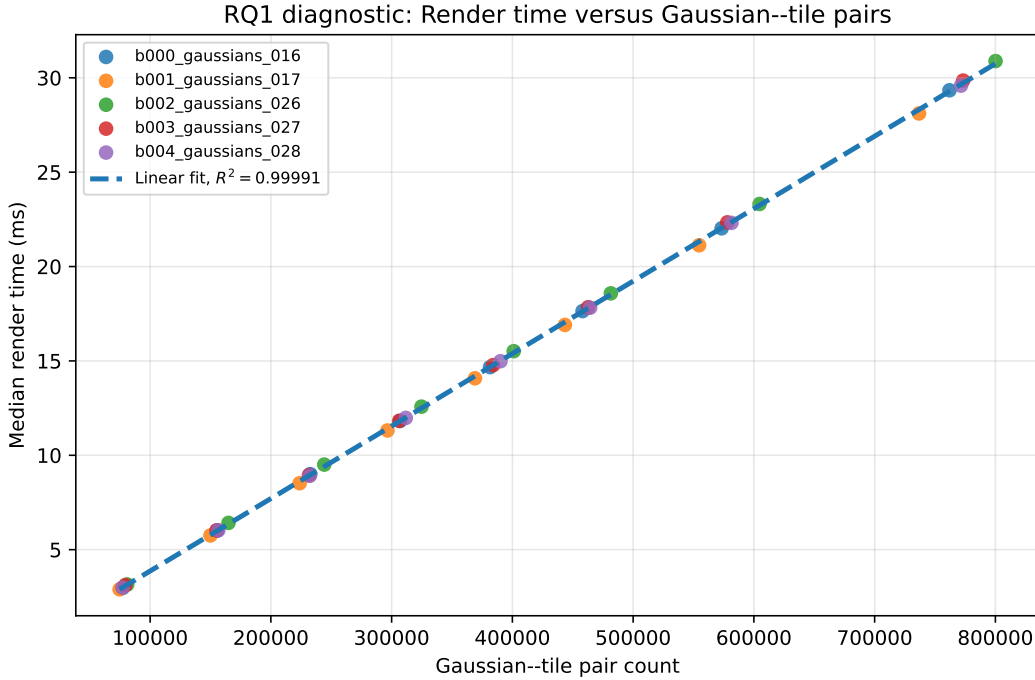


Figure 3: Relationship between Gaussian–tile pair count and median rendering time across all blocks and payload levels. Each marker represents one block at one payload ratio after aggregation across five timing repetitions. The fitted regression line demonstrates that projected tile coverage explains residual block-specific performance differences not captured by Gaussian count alone.

Figure 4 compares the five blocks when all 50,000 Gaussians are active.

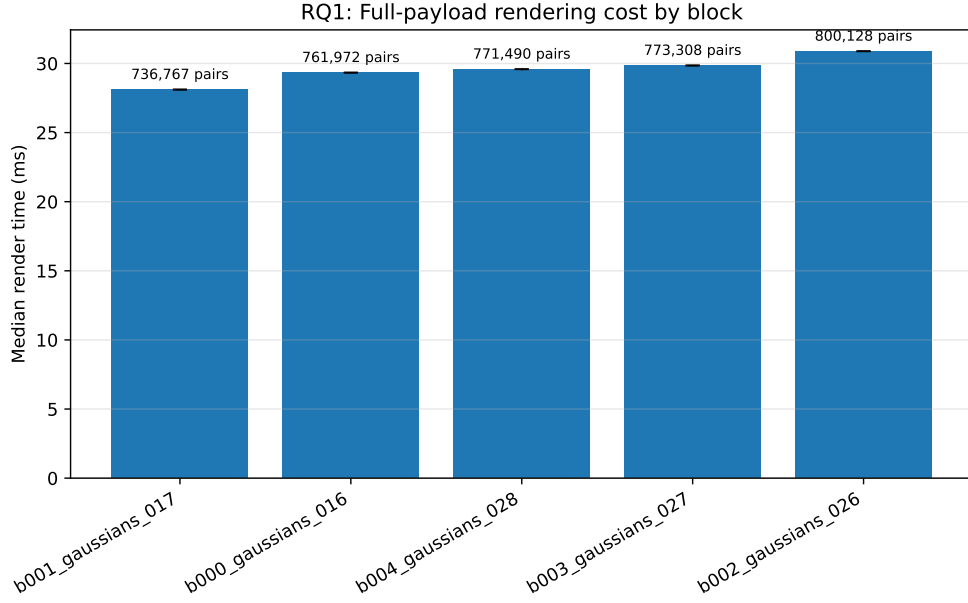


Figure 4: Full-payload median rendering time for the five pretrained blocks. All blocks contain 50,000 active Gaussians, but their rendering costs differ because their Gaussian–tile pair counts and projected spatial coverage are not identical.

1.1.10 Scalability Models

A separate linear model was fitted to each block:

$$T_{\text{render}}^{(b)} = \beta_0^{(b)} + \beta_1^{(b)} N_{\text{active}} + \epsilon. \quad (13)$$

The resulting slopes are shown in Table 4.

Table 4: Block-specific linear scalability models.

Block	Additional Time per 1,000 Gaussians (ms)	R^2
gaussians_016	0.584	0.99996
gaussians_017	0.560	0.99999
gaussians_026	0.614	0.99992
gaussians_027	0.594	0.99991
gaussians_028	0.591	0.99994

All five blocks exhibited an almost perfectly linear relationship between active Gaussian count and rendering time, with block-specific R^2 values greater than 0.9999.

Across the blocks, the rendering-time increase ranged from approximately

$$0.560 \text{ ms} \quad (14)$$

to

$$0.614 \text{ ms} \quad (15)$$

for every additional 1,000 Gaussians.

A pooled linear model with block-specific intercepts produced

$$T_{\text{render}} \approx 0.019 + 5.8871 \times 10^{-4} N_{\text{active}} + \gamma_b, \quad (16)$$

where `gaussians_016` is the reference block and γ_b represents the block-specific intercept adjustment.

The pooled model achieved

$$R^2 \approx 0.99905 \quad (17)$$

with an RMSE of approximately

$$0.253 \text{ ms.} \quad (18)$$

The fitted common slope corresponds to approximately

$$0.589 \text{ ms} \quad (19)$$

per additional 1,000 active Gaussians.

Figure 5 compares the relative scaling behaviour of the five blocks after normalising each block by its own full-payload rendering time.

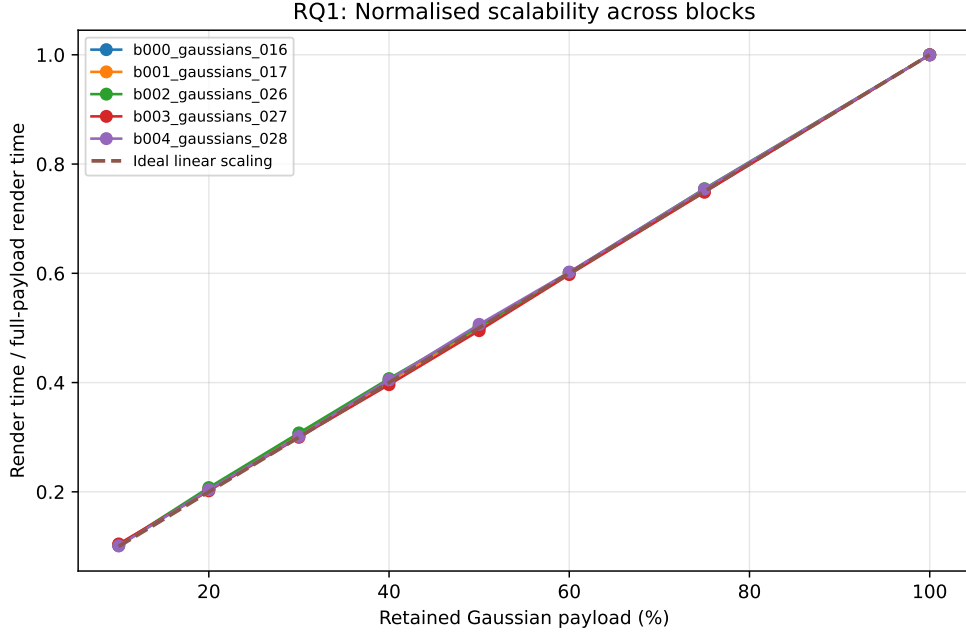


Figure 5: Render time normalised by the full-payload render time for each block. The close alignment of the block curves indicates that the relative payload-scaling behaviour is consistent across the five pretrained representations. The dashed reference indicates ideal linear scaling with retained Gaussian percentage.

1.1.11 Qualitative Reconstruction Check

Figure 6 presents representative rendering outputs from the five pretrained blocks at selected Gaussian retention ratios alongside the corresponding ground-truth (GT) MIPs and pixelwise absolute differences. Each GT volume is normalised with the same per-block minimum and maximum used during training, then rendered with the benchmark camera geometry, a 90° vertical field of view, and 64 depth samples. GT and reconstruction panels share a linear grayscale response scale; the $|I_{\text{render}} - I_{\text{GT}}|$ panels share a separate linear error scale.

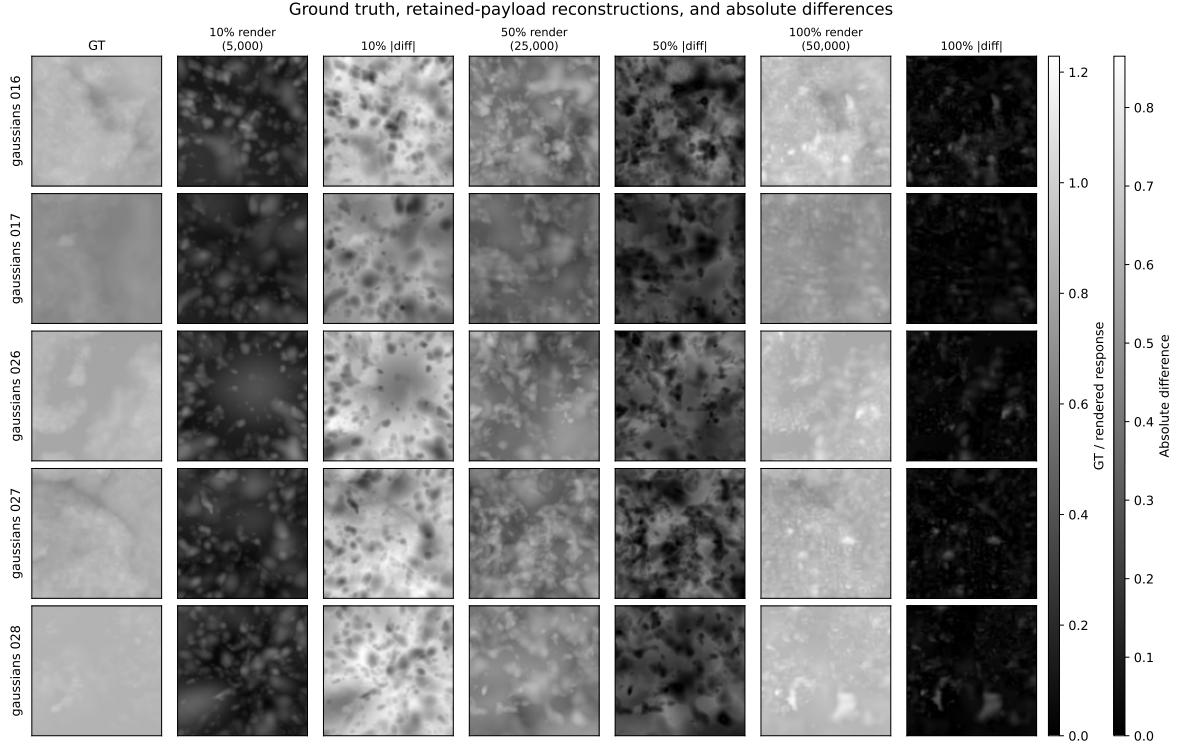


Figure 6: Ground-truth MIPs, retained-payload reconstructions, and pixelwise absolute differences for five pretrained blocks at 10%, 50%, and 100% Gaussian retention. Each row represents one block. GT and reconstruction panels share the response scale, while all difference panels share the separate error scale.

1.1.12 Cross-Block Variation

Variation between blocks increased moderately as the Gaussian payload increased. At the 10% payload, the standard deviation of median FPS across blocks was approximately 11.66 FPS. At the complete payload, the corresponding standard deviation was approximately 1.15 FPS.

The absolute FPS spread became smaller at high payloads because FPS is inversely related to frame time. However, block-specific differences remained visible in rendering time and Gaussian-tile pair count.

The full-payload frame times ranged from approximately

$$28.11 \text{ ms} \tag{20}$$

to

$$30.89 \text{ ms.} \tag{21}$$

This corresponds to a difference of approximately 9.9% between the fastest and slowest blocks despite identical Gaussian counts and payload sizes.

Consequently, active Gaussian count is a strong first-order predictor of rendering cost, while projected tile coverage explains the residual block-to-block variation.

1.1.13 Repeatability

The five timing repetitions for most configurations exhibited very low variation. At the complete payload, the fifth-percentile FPS was nearly equal to the median FPS for every block. Across all five blocks, the mean median FPS was 33.87, while the mean fifth-percentile FPS was 33.86.

This small difference indicates that steady-state rendering performance was temporally stable after the warm-up period.

A limited number of early low-payload repetitions contained slightly slower frames, but these deviations did not affect the monotonic payload trend or the fitted scalability models.

1.1.14 Answer to RQ1

The multi-block results show that active Gaussian payload has a strong, monotonic, and approximately linear effect on steady-state rendering time on the fixed GPU.

For every evaluated block,

$$T_{\text{render}} \propto N_{\text{active}}. \quad (22)$$

Across the five blocks, each additional 1,000 active Gaussians increased frame time by approximately 0.560–0.614 ms. As a consequence, FPS decreased reciprocally as the payload increased.

The experiment also showed that blocks with equal Gaussian counts can have different rendering costs because their Gaussian–tile pair counts differ. Therefore, Gaussian count determines the dominant scalability trend, while the projected spatial coverage of the Gaussians explains the smaller block-specific deviations.

The answer to RQ1 is therefore:

Across five pretrained blocks rendered on the same GPU under identical settings, steady-state rendering time increased approximately linearly with active Gaussian payload. The dominant scaling factor was the number of active Gaussians, while Gaussian–tile pair count accounted for residual performance differences between blocks. Consequently, FPS decreased approximately reciprocally as the active payload increased.